Bank Loan Approval & Performance Tracking System

Executive Summary

In the modern banking ecosystem, data-driven decision-making is critical for minimizing credit risk, improving loan approval accuracy, and ensuring regulatory compliance. This project presents an **end-to-end Database Management System (DBMS) solution** for a *Bank Loan Approval & Performance Tracking System* built using **PostgreSQL**.

The system ingests raw loan application data, applies structured data cleaning and validation techniques, enforces business constraints, and enables advanced analytical queries to generate actionable business insights. Unlike traditional academic DBMS projects, this solution mirrors **real-world banking workflows**, focusing on risk assessment, approval optimization, and performance monitoring.

This project is designed to be **industry-ready for 2026–2027 roles** in Data Analytics, Business Intelligence, and Backend Data Engineering.

1. Business Problem Statement

Banks process thousands of loan applications daily. Poor data quality, inconsistent records, and lack of analytical visibility can lead to: - Higher default risk - Biased or incorrect loan approvals - Inefficient credit decision-making - Revenue leakage

Objective: To design a centralized database system that: - Stores loan application data in a structured format - Cleans and validates inconsistent real-world data - Tracks loan approval patterns and risk indicators - Enables business-driven insights using SQL

2. Project Objectives

- Design a robust relational database schema for loan analytics
- Perform professional-grade data cleaning and preprocessing
- Apply core and advanced SQL concepts used in industry
- Generate meaningful business insights for banking decisions
- Optimize database performance using indexing and transactions

3. Dataset Overview (Real-World Context)

Source: Kaggle – Loan Prediction Dataset

Link: https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset

The dataset represents anonymized loan application records collected by a financial institution. Each row corresponds to a unique loan request submitted by a customer.

Key Attributes Explained

- **Loan_ID:** Unique identifier for each loan application
- **ApplicantIncome / CoapplicantIncome:** Monthly income sources
- **Credit_History:** Indicator of past credit repayment behavior
- **LoanAmount & Loan_Amount_Term:** Requested loan size and tenure
- **Property_Area:** Geographic risk segmentation (Urban, Semiurban, Rural)
- **Loan_Status:** Final approval decision (Approved / Rejected)

This dataset closely resembles data used by banks for **credit scoring and underwriting**.

4. Database Architecture & Schema Design

4.1 Staging Table Strategy

A **loan_staging** table was created to ingest raw data directly from CSV files. This mirrors real-world ETL pipelines where raw data is first staged before normalization.

Key design considerations: - Use of **CHECK constraints** to enforce business rules - Use of **PRIMARY KEY** to prevent duplicate loan records - Numeric validations for income and loan amount fields

4.2 Normalized Structure

To demonstrate enterprise DBMS practices, the dataset was later decomposed into: - **customers table** – demographic and employment details - **loans table** – loan-specific financial attributes

This ensures: - Reduced redundancy - Better data integrity - Scalable schema design

5. Data Cleaning & Preprocessing Strategy

Real-world banking data is rarely clean. The following professional techniques were applied:

5.1 Missing Value Treatment

- **Categorical columns** (Gender, Married, Self_Employed):
    - Filled using **Mode (most frequent value)**

- **Numerical columns** (LoanAmount):
    - Filled using **Average (Mean)** to preserve distribution

5.2 Data Standardization

- Standardized dependent categories (e.g., converting 3+ to 3)
- Ensured consistent data types across columns

5.3 Duplicate Removal

- Duplicate Loan IDs removed using PostgreSQL system column **ctid**
- Retained earliest valid record per loan

5.4 Post-Cleaning Validation

- NULL audits using filtered COUNT queries
- Constraint checks to ensure business rule compliance

This approach reflects **industry ETL best practices**.

6. SQL Concepts Demonstrated (Recruiter-Focused)

SQL Concept	Real-World Application
Constraints	Enforcing banking rules (income ≥ 0, valid credit history)
Aggregate Functions	Approval rate, income distribution analysis
Joins	Customer-loan relationship modeling
Subqueries	Above-average risk and income analysis
Views	Reporting layer for BI dashboards
Indexing	Performance optimization on frequent filters
Transactions	ACID-compliant loan updates
Case Statements	Income and loan segmentation

This mapping makes the project **interview-friendly and job-relevant**.

7. Business Insights & Analytical Findings

Key Insights Generated

- Applicants with **valid credit history** have significantly higher approval rates
- **Semiurban and Urban regions** show safer lending patterns
- Middle-income applicants demonstrate the **best approval-to-risk balance**
- Self-employed applicants face slightly lower approval probabilities

- Graduates receive **higher average loan amounts**, indicating trust-based lending
- Shorter loan terms show better repayment confidence

These insights directly support **credit policy optimization**.

8. Business Value to Banks

This system enables banks to: - Reduce credit default risk - Improve loan approval transparency - Identify high-risk customer segments early - Support data-backed lending strategies - Build BI dashboards on top of validated SQL views

10. Conclusion

This project demonstrates the ability to design, clean, optimize, and analyze a real-world banking database using PostgreSQL. It goes beyond academic SQL by emphasizing **business impact, data quality, and performance optimization**.